NYU Tandon School of Engineering CS2413/Fall 2025/Prof. Lisa Hellerstein

Name (John Doe, PRINT): NetID (`xyz9999`):

Exam 1. Tuesday, Oct. 21, 2025; 12:30-1:50pm.

- Do not look inside the exam until instructed.
- **Read** all the rules carefully.
- Put your **name** and **NetID** in the spaces above, and **sign** below.
- Write clearly and concisely, **IN THE SPACES PROVIDED**.
- You have 1 hour and 20 minutes.
- Closed books, closed notes. Pen/pencil. Eraser. No phones. No electronics.
- Be clear. If I cannot read what you wrote, you will get no points for it.
- When giving a bound in asymptotic notation, do not include lower order terms, multiplicative constants, or the constant base of a logarithm. For example, write $O(n \log n)$ rather than $O(5n \log_7 n + 2)$
- You will get no credit for any answer where you contradict yourself.
- Read every question *completely* before answering. You risk getting no points if you answer a wrong question.
- If you need more space, use the extra pages at the end; write problem number there; write "see extra pages" in place of your answer, or we may not look there.
- **Do not write your answers on the back side of the pages; they won't be scanned.**

Note: The 2-page "formula sheet" is at the end of the exam.

- I, the student, read and understood the rules.

  Signature:

1. (10 points) Short answer questions. **For each of the following, give your answer in theta notation, as a function of n.** Don't forget to simplify the expression inside the theta notation, so it has no lower order terms, multiplicative constants, or constant bases for logarithms.

   (a) Worst-case running time of heapsort on an array of size n, in theta notation, is

   (a) _____

   (b) Sum of the numbers from 1 to n^2 (as a function of n), in theta notation, is

   (b) _____

(c) Number of stars printed by the following pseudocode, in theta notation:

```
for i=1 to n
     for j = n downto i+1
          print "*"
```

(c) _____

(d) Sum of the heights of all the nodes in a max-heap of size n, in theta notation. Recall that the height of a node in a heap is the maximum number of edges on a simple path from that node down to a leaf node (so, e.g., the height of each leaf node is 0).

(d) _____

(e) Expected number of iterations of the while loop below, as a function of n, in theta notation:

```
continue = true
while continue == true
     choose a number i uniformly at random between 1 and n (inclusive)

     choose a number j uniformly at random between 1 and n (inclusive)

     if i==j
         continue = false
```

(e) _____

2. (4 points) Consider the recursion tree for the following recurrence. Assume n is a power of 2.

$$T(n) = 5T(n/2) + 4n \text{ for } n > 1$$

$$T(1) = 3$$

Suppose that, as usual, we number the levels of the recursion tree starting at 0. You do not have to solve this recurrence. Just answer the questions below.

(a) As a function of i, how many nodes are at level i of the tree? Give an exact expression. Do not use asymptotic notation in your answer.

(a) _____

(b) As a function of n and i, what is the "input size" for the nodes at level i? Give an exact expression. Do not use asymptotic notation in your answer.

(b) _____

(c) As a function of n, what is the level number at the leaves? Give an exact expression. Do not use asymptotic notation in your answer.

(c) _____

(d) What is the total amount of "local work" at all the leaf nodes? Give an exact expression. Do not use asymptotic notation in your answer.

(d) _____

3. (6 points) For each of the following, indicate whether $f(n) = o(g(n))$ (little-oh), $f(n) = \omega(g(n))$ (little-omega), or $f = \theta(g(n))$. No justification is needed.

   (a) $f(n) = 5\log_2(n!), \quad g(n) = n\log_2 n$

   (a) _____

   (b) $f(n) = 2^n, \quad g(n) = n!$

   (b) _____

   (c) $f(n) = \sum_{i=1}^{n} \frac{n}{i}, \quad g(n) = n$

   (c) _____

4. Consider the following Index-Finding Problem. You are given as input two *unsorted* arrays of integers: $A[1...N]$ of length N, and $B[1...2^N]$ of length 2^N. The elements of A are all distinct. The elements of B are not necessarily distinct. For each index i of array B, you need to determine whether the value $B[i]$ appears in array A. If so, you need to determine the position where it appears. More specifically, you need to compute an output array $D[1...2^N]$ where for $i \in \{1, \ldots, 2^N\}$, $D[i] = 0$ if $B[i]$ doesn't appear in A, otherwise $D[i]$ equals the position in array A that contains the value $B[i]$.

   For example, if $A = [3, 5]$ and $B = [5, 2, 5, 3]$, then $D = [2, 0, 2, 1]$.

   (a) (2 points) We would like to lower bound the number of comparisons performed by any comparison-based algorithm for this Index-Finding problem. (A comparison-based algorithm for this problem can compare two elements of A to each other, two elements of B to each other, or an element of A to an element of B.) To get the lower bound, first answer the following question: As a function of N, how many possible values are there for output array $D = [1 \ldots 2^N]$? Give an exact expression, do not use asymptotic notation.

   (a) _____

   (b) (4 points) Use your answer to part (a) to complete the following proof of a lower bound on the worst-case number of comparisons made by any comparison-based algorithm for solving this Index-Finding problem. Give exact expressions, do not use asymptotic notation. If you use logarithms in your expression, make sure to include the base of the logarithm.

   Consider a comparison-based algorithm solving this problem.
   The comparisons it makes can be represented by a binary decision tree.
   It follows from part (a) that the number of leaves of this decision tree is at least

   i. _____

   Since the decision tree is a binary tree, to have that many leaves its height must be at least

   ii. _____

   Therefore, the worst-case number of comparisons made by any comparison-based algorithm for this problem is at least

   iii. _____

(c) (4 points) Briefly but clearly describe a comparison-based algorithm for this Index-Finding Problem that makes $\theta(2^N \log N)$ comparisons in the worst case. You may use algorithms from class in your description, without explaining how they work. For example, you can say "Run Karatsuba's algorithm" without describing how it works. Remember that input arrays A and B are *unsorted*.

5. (2 points) Suppose we have a file with student names. Each row of the file has the name of a student, divided into two columns, one for the student's first name and one for the student's last name. For example:

```
Isaac          Newton
Taylor         Swift
Donald         Knuth
Anatoly        Karatsuba
Jonathan       Swift
...
```

This question is about Radix Sort, which will not be covered in Fall 2026 Exam 1.

We would like to sort the names in the file in alphabetical order by last name, so that e.g., Anatoly Karatsuba comes before Isaac Newton. If two students have the same last name, then we want their names to be sorted in alphabetical order of the first names, so that e.g., Jonathan Swift appears before Taylor Swift.

Suppose we can perform the following two operations on the file: (1) sort the names alphabetically by First Name only, by doing a sort on the first column, (2) sort the names alphabetically by Last Name only, by doing a sort on the second column. Assume that these sorts are stable.

Briefly but clearly describe an easy way to use these two operations to produce the desired sorted version of the file.

6. Suppose we are sorting an array $A[1 \ldots n]$ which contains elements of the form $(item, price)$, where **the key of each element is the price** (in dollars). Here is a version of insertion sort for such an array.

```
for i=2 to n
     x = A[i]
     j = i-1
     while j > 0 and A[j].key > x.key
          A[j+1] = A[j]
          j=j-1
     A[j+1] = x
```

(a) (2 points) Suppose we run this version of insertion sort on input array

$$A = [(coat, 76), (hat, 34), (cup, 15), (shirt, 34)]$$

What is the resulting value of A?

(b) (1 point) Is the above version of insertion sort stable? Answer yes or no.

(b) _____

Parts (b) and (c) concern the definition of a stable sorting algorithm, which will not be covered in the Fall 2026 Exam 1.

(c) (1 point) Suppose that in the above pseudocode, we changed the line with the while statement to say

```
while j > 0 and A[j].key >= x.key
```

Is this modified version of insertion sort stable? Answer yes or no.

(c) _____

7. (5 points) Consider the following recurrence, defined for integers $n \geq 1$:

$$T(n) = 2T(\lfloor \frac{2n}{3} \rfloor) + n^2 \text{ for } n \geq 2$$
$$T(1) = 7$$

Use proof by induction to prove that $T(n) = O(n^2)$. More particularly, prove by induction that $T(n) \leq cn^2$ for all $n \geq 1$, for some integer $c > 0$. Do not use the master method. Do not use a recursion tree.

Make sure that you do the following in your proof:

- Include the base case, where $n = 1$ (this is the only base case)
- Include an inductive assumption and an inductive step (you can do these separately or together)
- Give a specific value for c that makes your proof "work"

The following page has more room for your answer, if this page isn't enough.

This isn't the last page of the exam.
Go to the next page.

8. For each of the following recursive algorithms, first give the recurrence expressing the worst-case running time of the algorithm (ignore ceilings and floors). You do not need to include the base case or floors and ceilings. Then give the worst-case running time of the algorithm, in θ-notation. `The answer to part (a) is given to you as an example.`

   (a) Worst-case running time of (the recursive version of) BINARY SEARCH.

   i. Recurrence for worst-case running time is

   i. $T(n) = T(n/2) + \theta(1)$

   ii. Worst-case running time in θ notation is

   ii. $\theta(\log n)$

   (b) (3 points) Worst-case running time of a variant of MERGESORT, which we call SLOWMERGESORT. It replaces the linear-time merge procedure used by standard MERGESORT with a slow merge procedure that runs in time $\theta(n^{1.5})$. SLOWMERGESORT still divides the array in half in each recursive call.

   i. Recurrence for worst-case running time is

   i. _____

   ii. Worst-case running time in θ notation is

   ii. _____

   (c) (3 points) Worst-case running time of a variant of QUICKSORT, which we call MMQUICKSORT. To choose the pivot, it uses the Median of Medians algorithm to find the kth order statistic of the array, for $k = \lfloor (n+1)/2 \rfloor$. That is, it uses the Median of Medians algorithm to find the median value in the input array. It then partitions around that pivot, and recurses on Low and High, as usual.

   i. Recurrence for worst-case running time is

   i. _____

   ii. Worst-case running time in θ notation is

   ii. _____

The median of medians algorithm will not be covered in the Fall 2026 Exam 1

9. (3 points) Let $T(n)$ be defined as follows:

$$T(n) = \sum_{i=0}^{2\log_3 n} 3^i$$

Which of the following is a correct asymptotic expression for $T(n)$?

- ◯ $\theta(1)$
- ◯ $\theta(\log n)$
- ◯ $\theta(\sqrt{n})$
- ◯ $\theta(n)$
- ◯ $\theta(n \log n)$
- ◯ $\theta(n^2)$
- ◯ $\theta(n^4)$

Explain/justify. No credit without a justification.

Extra space. Write **question number** and your answer in the box.

Extra space. Write **question number** and your answer in the box.

Formula Pages for Exam 1, Oct. 21

Theorem 0.1 (Our Master Theorem)**.** *Let a, b, d, n_0 be constants such that $a > 0, b > 1, d \geq 0$ and $n_0 > 0$.*
Let $T(n) = aT(n/b) + \theta(n^d)$ for when $n \geq n_0$, and $T(n) = \theta(1)$ when $0 \leq n < n_0$. Then,

$$T(n) = \begin{cases} \theta(n^d \log n) & \text{if } d = \log_b a \\ \theta(n^{\log_b a}) & \text{if } d < \log_b a \\ \theta(n^d) & \text{if } d > log_b a \end{cases}$$

We assume here that $T(n)$ is a function defined on the natural numbers. We use $aT(n/b)$ to mean $a'T(\lfloor n/b \rfloor) + a''(T(\lceil n/b \rceil)$ where $a', a'' > 0$ such that $a' + a'' = a$.

Logarithms

We use the following notations:

$$\begin{aligned} \lg n &= \log_2 n && \text{(binary logarithm)}, \\ \ln n &= \log_e n && \text{(natural logarithm)}, \\ \lg^k n &= (\lg n)^k && \text{(exponentiation)}, \\ \lg \lg n &= \lg(\lg n) && \text{(composition)}. \end{aligned}$$

We adopt the following notational convention: in the absence of parentheses, *a logarithm function applies only to the next term in the formula*, so that $\lg n + 1$ means $(\lg n) + 1$ and not $\lg(n + 1)$.

For any constant $b > 1$, the function $\log_b n$ is undefined if $n \leq 0$, strictly increasing if $n > 0$, negative if $0 < n < 1$, positive if $n > 1$, and 0 if $n = 1$. For all real $a > 0, b > 0, c > 0$, and n, we have

$$a = b^{\log_b a}, \tag{3.17}$$

$$\log_c(ab) = \log_c a + \log_c b, \tag{3.18}$$

$$\log_b a^n = n \log_b a,$$

$$\log_b a = \frac{\log_c a}{\log_c b}, \tag{3.19}$$

$$\log_b(1/a) = -\log_b a, \tag{3.20}$$

$$\log_b a = \frac{1}{\log_a b},$$

$$a^{\log_b c} = c^{\log_b a}, \tag{3.21}$$

where, in each equation above, logarithm bases are not 1.

[from CLRS p. 66]

Geometric series

When r is a real number such that $r \neq 1$,

$$\sum_{k=0}^{n} r^k = 1 + r + r^2 + \ldots r^n = \frac{r^{n+1} - 1}{r - 1}$$

When r is a real number such that $|r| < 1$,

$$\sum_{k=0}^{\infty} r^k = \frac{1}{1 - r}$$

Harmonic numbers

For n a positive integer, the nth Harmonic number is

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots + \frac{1}{n} = \theta(\log n)$$

In fact,

$$\ln(n+1) \leq H_n \leq \ln n + 1$$